



武汉大学
Wuhan University

第四章 组合逻辑电路

4.4 常用组合逻辑电路

武汉大学计算机学院 2025-2026第一学期



主要内容

Main Content

加法器

编码器

译码器

4.4.1 加法器

- **加法器**是实现二进制数加法运算的逻辑电路；
- 计算机中加、减、乘、除等运算都是转换为若干步加法运算实现的
- 半加器：只考虑加数和被加数，不考虑低位进位的逻辑部件
- 全加器：同时考虑加数、被加数、低位进位的逻辑部件
- 串行加法器：低位加法完成，进位信号产生后，相邻高位加法运算才能进行
- 并行加法器：提前产生进位信号，高位加法不必等到低位加法完成后再进行

4.4.1 加法器

1. 半加器

(1) 半加器真值表

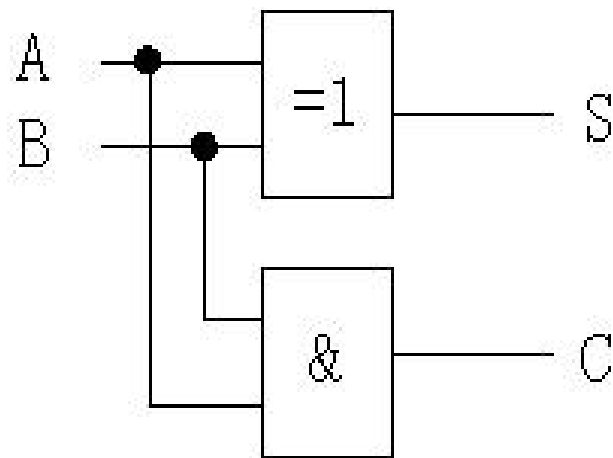
输入		输出	
被加数A	加数B	和S	进位C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

(2) 逻辑函数

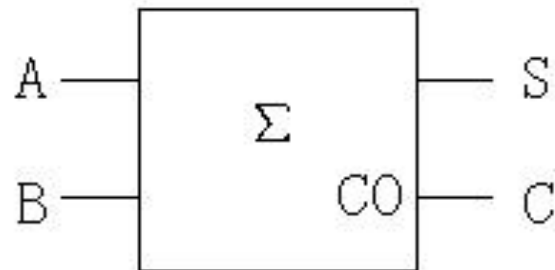
$$S = \overline{A}B + A\overline{B} = A \oplus B$$

$$C = AB$$

(3) 逻辑图



(4) 逻辑符号



4.4.1 加法器

2. 全加器

1位全加器的逻辑函数设计

(1) 全加器真值表

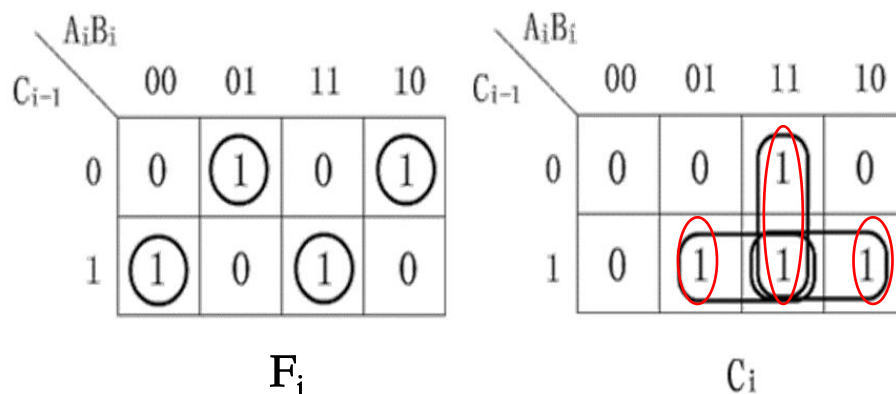
输入			输出	
A_i	B_i	C_{i-1}	F_i	C_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

(2) 逻辑函数最小项表达式

$$F_i(A_i, B_i, C_{i-1}) = \sum m(1, 2, 4, 7)$$

$$C_i(A_i, B_i, C_{i-1}) = \sum m(3, 5, 6, 7)$$

(3) 卡诺图化简



(4) 函数表达式

$$F_n = A_n \oplus B_n \oplus C_{n-1}$$

$$\begin{aligned} C_n &= (A_n + B_n)C_{n-1} + A_n B_n \\ &= (A_n \oplus B_n)C_{n-1} + A_n B_n \\ &= (A_n \oplus B_n)C_{n-1} \cdot A_n B_n \end{aligned}$$

C_n 中的第二步并不是第一步化简得到的，而是不同的圈法得到的。因为第二种圈法可以共用 F_n 中的异或门。

4.4.1 加法器

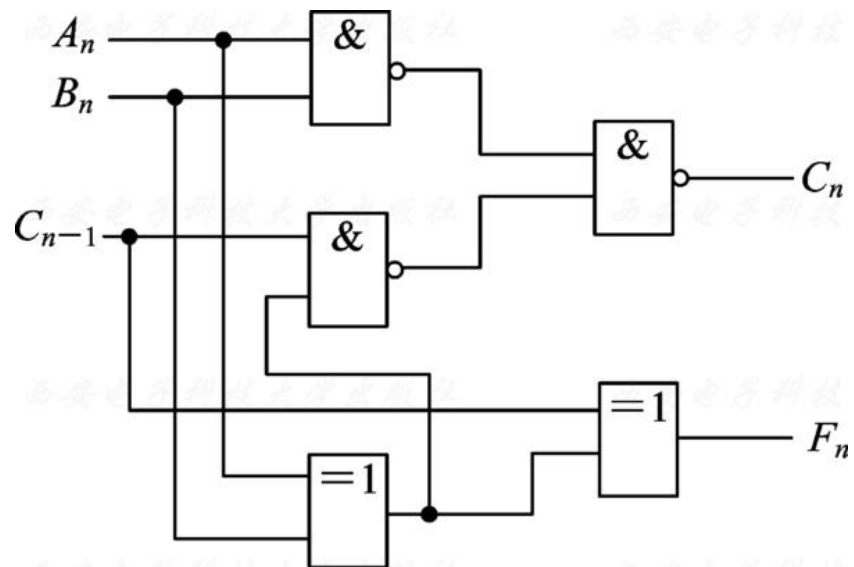
2. 全加器

1位全加器的逻辑函数设计

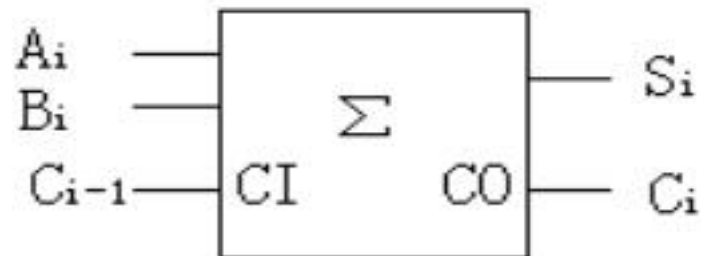
(1) 全加器真值表

输入			输出	
A_i	B_i	C_{i-1} 进位输入	F_i	C_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

(5) 逻辑电路图



(6) 逻辑符号



4.4.1 加法器

2. 全加器

1位全加器的Verilog HDL程序设计

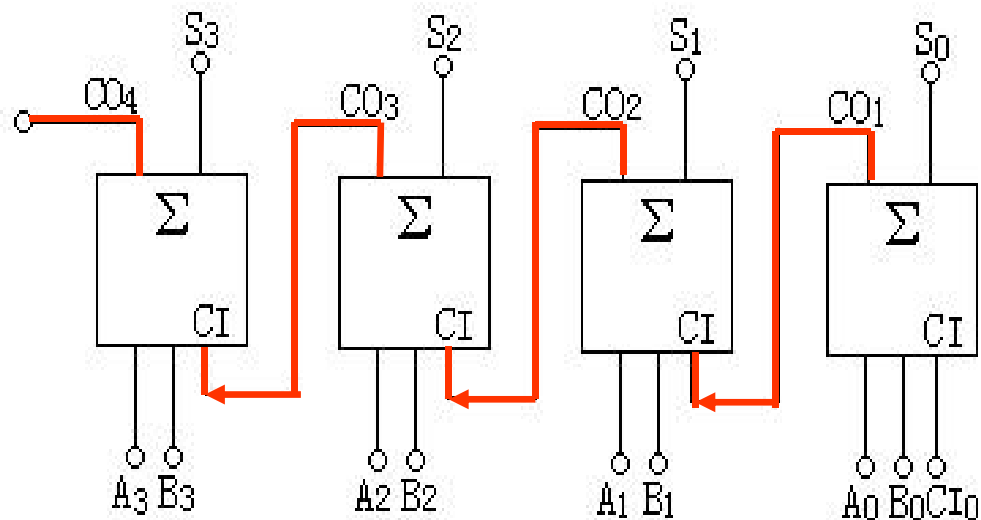
```
module samp4_4_1(A, B, C0, S, C1); //调用1位全加器的顶层模块
    input  A, B, C0;
    output S, C1;
    adder_full u1(.ia(A), .ib(B), .ic(C0), .os(S), .oc(C1));
    //调用1位全加器模块
```

```
endmodule
```

```
module adder_full(ia, ib, ic, os, oc); // 1位全加器模块
    input ia, ib, ic;
    output os, oc;
    assign {oc, os}=ia+ib+ic;
endmodule
```

4.4.1 加法器

3. 串行进位加法器



- 优点：电路容易实现
- 缺点：速度慢

4.4.1 加法器

4. 并行进位加法器

设加数A和被加数B均为四位二进制数，表示如下：

$$A = A_4 A_3 A_2 A_1 \quad B = B_4 B_3 B_2 B_1$$

设从低位到高位四个1位全加器的**进位输入信号**依次为：

$$C_0, C_1, C_2, C_3$$

则产生的**进位输出信号**依次为： C_1, C_2, C_3, C_4

4位并行全加器的逻辑函数：

$$\begin{array}{ll} S_1 = A_1 \oplus B_1 \oplus C_0 & C_1 = A_1 B_1 + C_0 (A_1 \oplus B_1) \\ S_2 = A_2 \oplus B_2 \oplus C_1 & C_2 = A_2 B_2 + C_1 (A_2 \oplus B_2) \\ S_3 = A_3 \oplus B_3 \oplus C_2 & C_3 = A_3 B_3 + C_2 (A_3 \oplus B_3) \\ S_4 = A_4 \oplus B_4 \oplus C_3 & C_4 = A_4 B_4 + C_3 (A_4 \oplus B_4) \end{array}$$

4.4.1 加法器

4. 并行进位加法器

$$\begin{array}{ll}
 S_1 = A_1 \oplus B_1 \oplus C_0 & C_1 = A_1 B_1 + C_0 (A_1 \oplus B_1) \\
 S_2 = A_2 \oplus B_2 \oplus C_1 & C_2 = A_2 B_2 + C_1 (A_2 \oplus B_2) \\
 S_3 = A_3 \oplus B_3 \oplus C_2 & C_3 = A_3 B_3 + C_2 (A_3 \oplus B_3) \\
 S_4 = A_4 \oplus B_4 \oplus C_3 & C_4 = A_4 B_4 + C_3 (A_4 \oplus B_4)
 \end{array}$$

设**进位生成项**为： $G_i = A_i B_i$ ， **进位传递项**为： $P_i = A_i \oplus B_i$ ， 则：

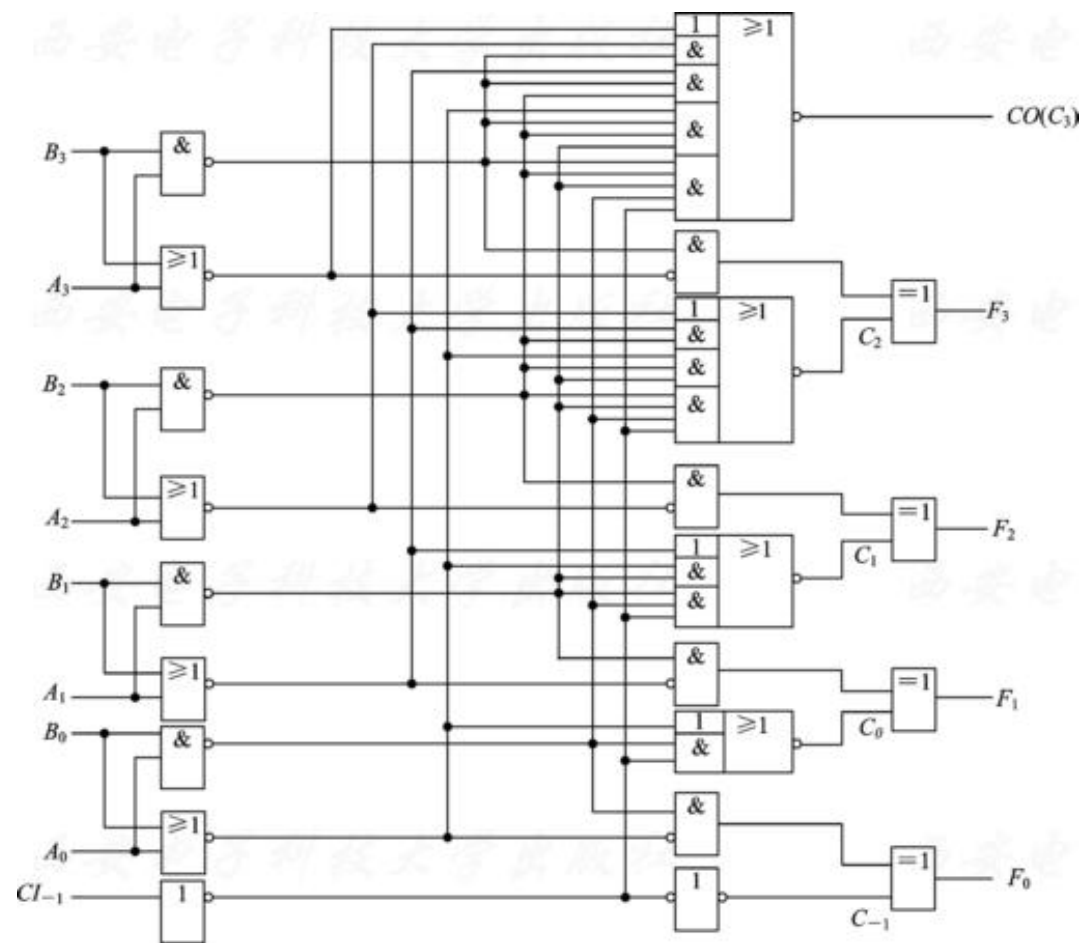
$$\begin{array}{l}
 C_1 = G_1 + P_1 C_0 \\
 C_2 = G_2 + P_2 C_1 = G_2 + P_2 G_1 + P_2 P_1 C_0 \\
 C_3 = G_3 + P_3 C_2 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 C_0 \\
 C_4 = G_4 + P_4 C_3 = G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 C_0
 \end{array}$$

输入相关： P_i 、 G_i 、 C_0

4.4.1 加法器

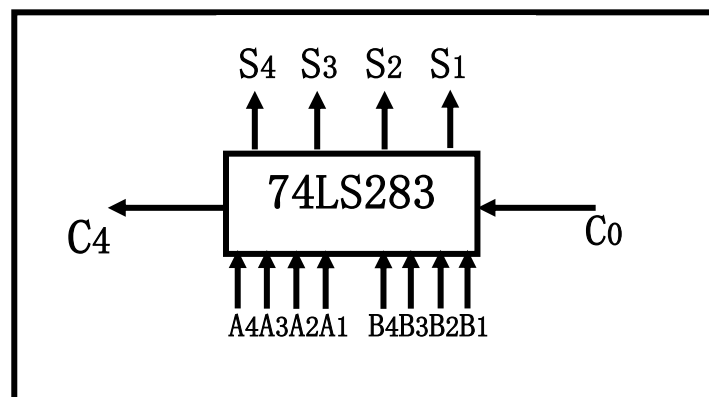
4. 并行进位加法器

➤ 逻辑电路图



➤ 逻辑符号

形成模块化



■ 优点：运算速度快

■ 缺点：电路结构较复杂

5. 加法器应用

➤ 例：试用一位全加器实现两位二进制乘法器。

解：设有两个2位二进制数分别为 $A=A_1A_0$ ， $B=B_1B_0$ ， P 是其乘法运算结果，则有：

$$P=A \times B=A_1A_0 \times B_1B_0$$

由于2位二进制的乘法运算的结果最多是4位二进制数，因此设 $P=P_3P_2P_1P_0$ ，其各位产生的计算过程如下：

$$\begin{array}{r}
 \begin{array}{cc} A_1 & A_0 \end{array} \\
 \times \quad \begin{array}{cc} B_1 & B_0 \end{array} \\
 \hline
 \begin{array}{cc} A_1B_0 & A_0B_0 \end{array} \\
 + \quad \begin{array}{cc} A_1B_1 & A_0B_1 \end{array} \\
 \hline
 \begin{array}{cccc} P_3 & P_2 & P_1 & P_0 \end{array}
 \end{array}$$

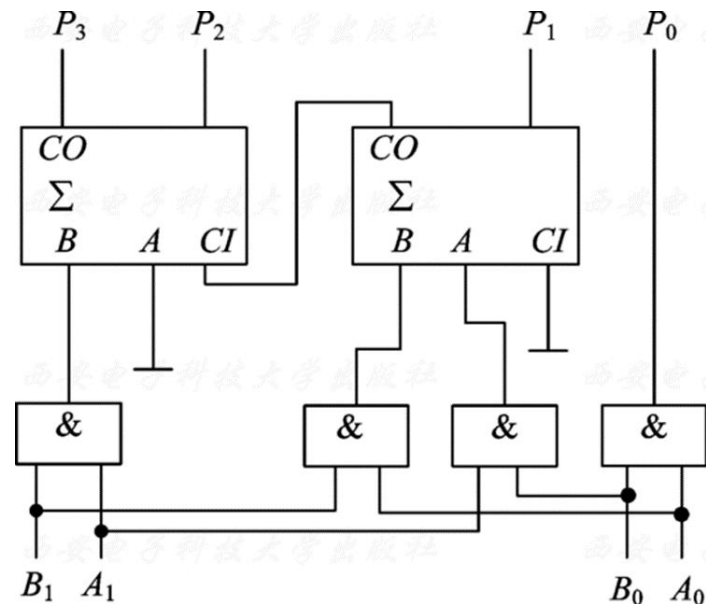
4.4.1 加法器

5. 加法器应用

其中：

$$\begin{cases} P_0 = A_0 B_0 \\ P_1 = A_1 B_0 + A_0 B_1 \\ P_2 = A_1 B_1 + C_1 \\ P_3 = C_2 \end{cases}$$

加法，不是“或”



其中， C_1 是 $A_1B_0 + A_0B_1$ 的进位输出， C_2 是 $A_1B_1 + C_1$ 的进位输出。

主要内容

Main Content

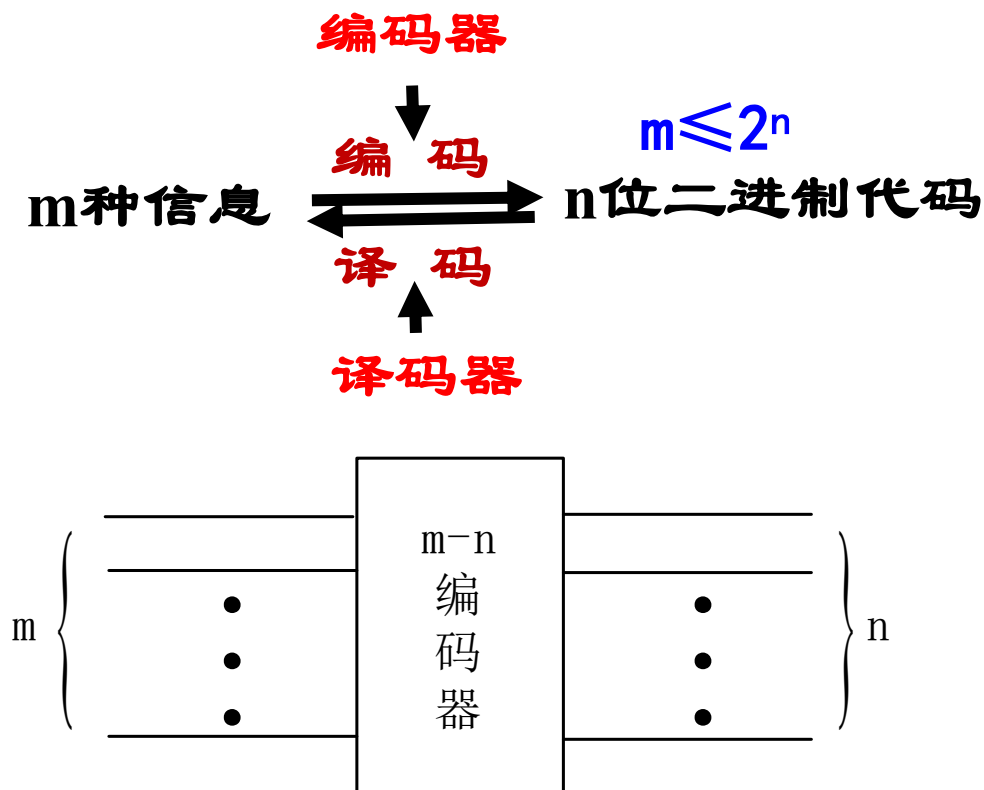
加法器

编码器

译码器

4.4.2 编码器

将一系列的输入信息（高、低电平）变换为以**二进制**按一定的规律编排的代码（多位输出信息），使**每组代码都对应一位有效输入信息**，这种功能称为**编码**。具有编码功能的逻辑电路称为**编码器**。



编码器的分类

◆ 按照输出的代码种类

- 二进制编码器 $m = 2^n$
- 二—十进制编码器 $m < 2^n$

◆ 按是否有优先权编码

- 普通编码器
- 优先编码器

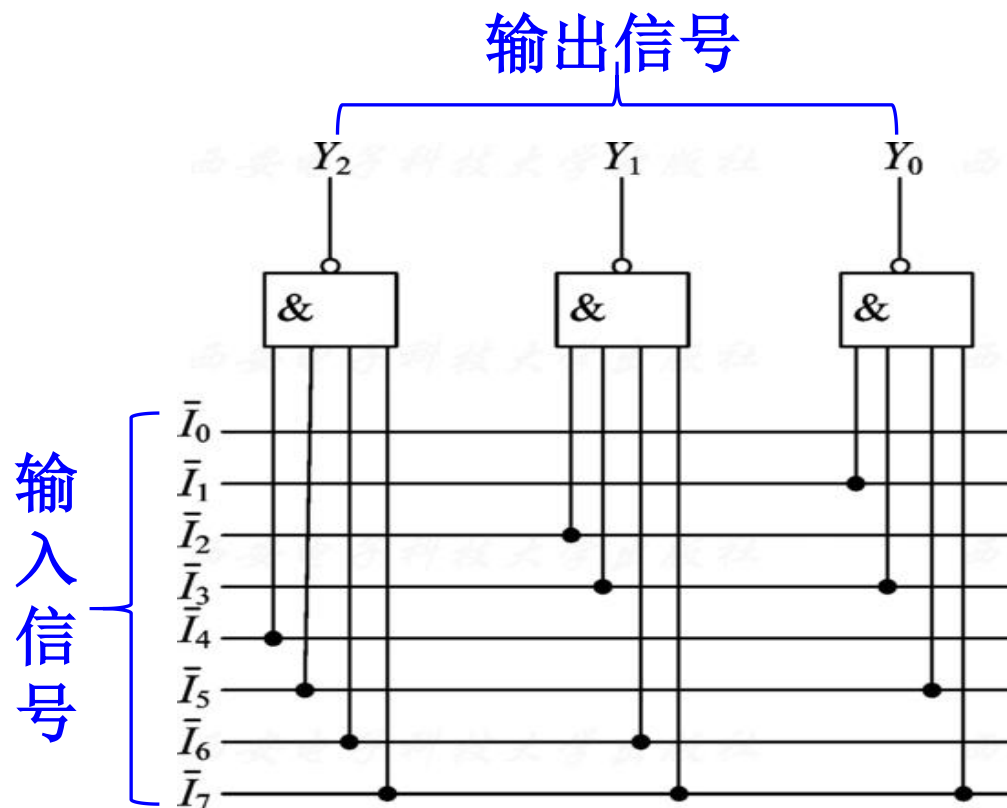
4.4.2 编码器

1. 普通编码器

8个输入信号： $\bar{I}_0 \sim \bar{I}_7$ (低电平有效)

3个输出信号：3位二进制代码 $Y_2Y_1Y_0$

8选1，对应一组输出编码



普通的8线-3线编码器

$$\begin{cases} Y_2 = \overline{\bar{I}_4 \bar{I}_5 \bar{I}_6 \bar{I}_7} \\ Y_1 = \overline{\bar{I}_2 \bar{I}_3 \bar{I}_6 \bar{I}_7} \\ Y_0 = \overline{\bar{I}_1 \bar{I}_3 \bar{I}_5 \bar{I}_7} \end{cases}$$

4.4.2 编码器

1. 普通编码器

普通8线—3线编码器输入输出关系表

输 入								输 出		
$\bar{I}_0$	$\bar{I}_1$	$\bar{I}_2$	$\bar{I}_3$	$\bar{I}_4$	$\bar{I}_5$	$\bar{I}_6$	$\bar{I}_7$	Y_2	Y_1	Y_0
0	1	1	1	1	1	1	1	0	0	0
1	0	1	1	1	1	1	1	0	0	1
1	1	0	1	1	1	1	1	0	1	0
1	1	1	0	1	1	1	1	0	1	1
1	1	1	1	0	1	1	1	1	0	0
1	1	1	1	1	0	1	1	1	0	1
1	1	1	1	1	1	0	1	1	1	0
1	1	1	1	1	1	1	0	1	1	1

每一组输入信号中只有一个有效信号

功能：

- 3位输出信号表示8个输入信号中的某一个有效输入的编号
- 任何时刻只允许所有输入中的一个有效，否则会出现输出混乱情况

1. 普通编码器

//普通8-3线编码器模块定义

```
module encoder1(ilN_N,oY_N);  
    input[7:0] ilN_N;  
    output reg [2:0] oY_N;  
    always@(ilN_N)  
        case(ilN_N)  
            8'b01111111: oY_N=3'b000;  
            8'b10111111: oY_N=3'b001;  
            8'b11011111: oY_N=3'b010;  
            8'b11101111: oY_N=3'b011;  
            8'b11110111: oY_N=3'b100;  
            8'b11111011: oY_N=3'b101;  
            8'b11111101: oY_N=3'b110;  
            8'b11111110: oY_N=3'b111;  
            default: oY_N=3'bxxx;  
        endcase  
endmodule
```

2. 优先编码器

优先编码器允许几个输入端**同时**加上有效信号，电路只对其中**优先级别最高的信号编码**（优先级别由设计者根据需要确定）

输入端：

8个输入信号： $\overline{I_0} \sim \overline{I_7}$ （优先权递增，低电平有效）

选通控制输入端： $\overline{ST}$

- 取值0：编码器的输出取决于输入信号（**编码电路工作**）
- 取值1：所有输出均被封锁为1（**编码电路不工作**）

输出端：

3位输出编码信号： $\overline{Y_2}, \overline{Y_1}, \overline{Y_0}$

选通输出端 Y_S **和扩展输出端** $\overline{Y_{EX}}$ ：用于电路的**扩展**

2. 优先编码器

优先编码器74LS148功能表

选通 输入	8位输入信号								3位输出信号			扩展 输出端	选通 输出端
$\overline{ST}$	$\overline{I_0}$	$\overline{I_1}$	$\overline{I_2}$	$\overline{I_3}$	$\overline{I_4}$	$\overline{I_5}$	$\overline{I_6}$	$\overline{I_7}$	$\overline{Y_2}$	$\overline{Y_1}$	$\overline{Y_0}$	$\overline{Y_{EX}}$	Y_s
1	Φ	Φ	Φ	Φ	Φ	Φ	Φ	Φ	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	Φ	Φ	Φ	Φ	Φ	Φ	Φ	0	0	0	0	0	1
0	Φ	Φ	Φ	Φ	Φ	Φ	0	1	0	0	1	0	1
0	Φ	Φ	Φ	Φ	Φ	0	1	1	0	1	0	0	1
0	Φ	Φ	Φ	Φ	0	1	1	1	0	1	1	0	1
0	Φ	Φ	Φ	0	1	1	1	1	1	0	0	0	1
0	Φ	Φ	0	1	1	1	1	1	1	0	1	0	1
0	Φ	0	1	1	1	1	1	1	1	1	0	0	1
0	0	1	1	1	1	1	1	1	1	1	1	0	1

- 当 $\overline{ST}=1$ 时，电路不工作，所有输出都为1；
- 当 $\overline{ST}=0$ ，且所有输入端均为1时，选通输出信号 $Y_s=0$ ，其余输出端均为1
- 当 $\overline{ST}=0$ ，且输入端包含有效信号时，三位输出信号为最高有效信号的编码（反码）；此时 $Y_s=1$ ，扩展输出信号 $\overline{Y_{EX}}=0$

2. 优先编码器

从高位到低位，按照优先级递减顺序，逐个检查是否为有效输入

```
module encoder_74148(iST_N,  
iIN_N, oY_N, oYEX_N, oYS);  
  
input iST_N;  
input [7:0] iIN_N;  
output reg [2:0] oY_N;  
output reg oYEX_N, oYS;  
  
always@(iST_N, iIN_N)  
  
if(!iST_N) //选通输入端有效  
begin  
    oYEX_N=0;  
    oYS=1;
```

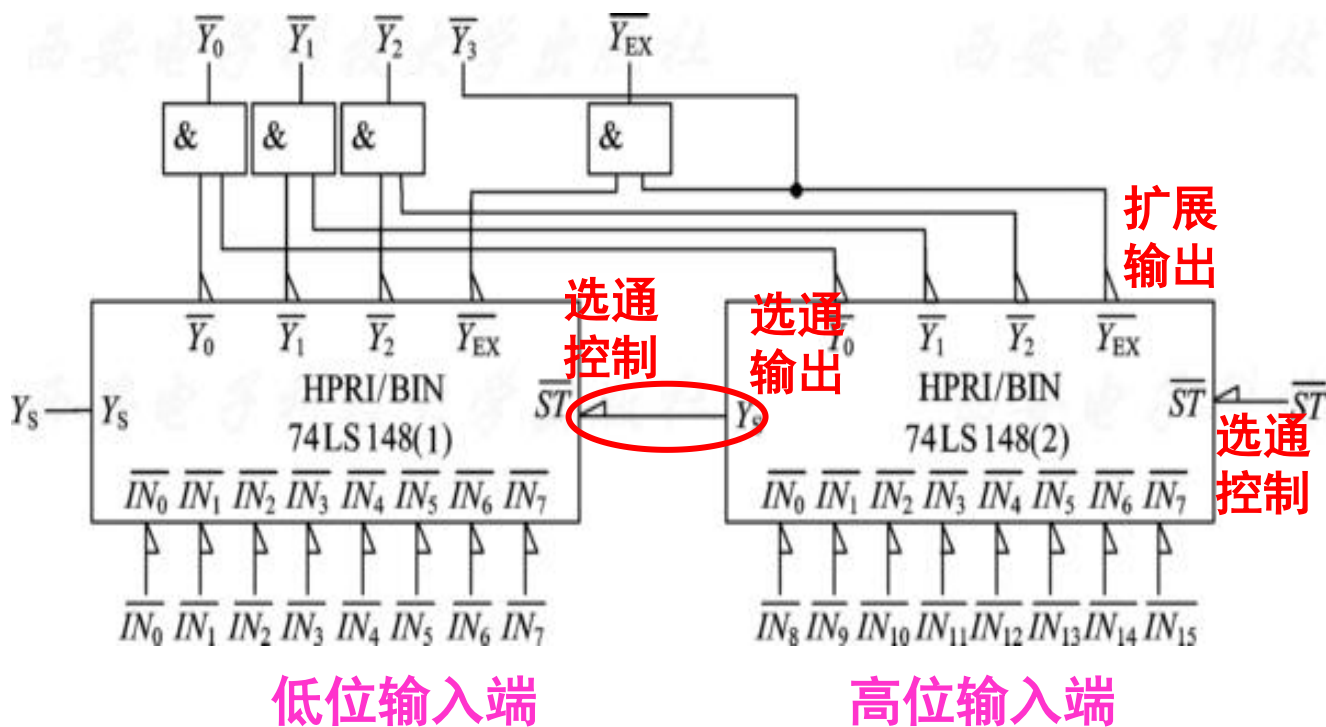
```
    if(iIN_N [7] ==0)  
        oY_N=3'h0;  
    else if(iIN_N [6] ==0)  
        oY_N=3'h1;  
    else if(iIN_N [5] ==0)  
        oY_N=3'h2;  
    else if(iIN_N [4] ==0)  
        oY_N=3'h3;  
    else if(iIN_N [3] ==0)  
        oY_N=3'h4;  
    else if(iIN_N [2] ==0)  
        oY_N=3'h5;  
    else if(iIN_N [1] ==0)  
        oY_N=3'h6;  
    else if(iIN_N [0] ==0)  
        oY_N=3'h7;
```

```
    else //无有效输入  
        begin  
            oY_N=3'h7;  
            oYEX_N=1;  
            oYS=0;  
        end  
    end  
    else //选通输入端无效  
        begin  
            oY_N=3'h7;  
            oYEX_N=1;  
            oYS=1;  
        end  
endmodule
```

4.4.2 编码器

2. 优先编码器

用8-3线优先编码器74LS148扩展成16-4线优先编码器。

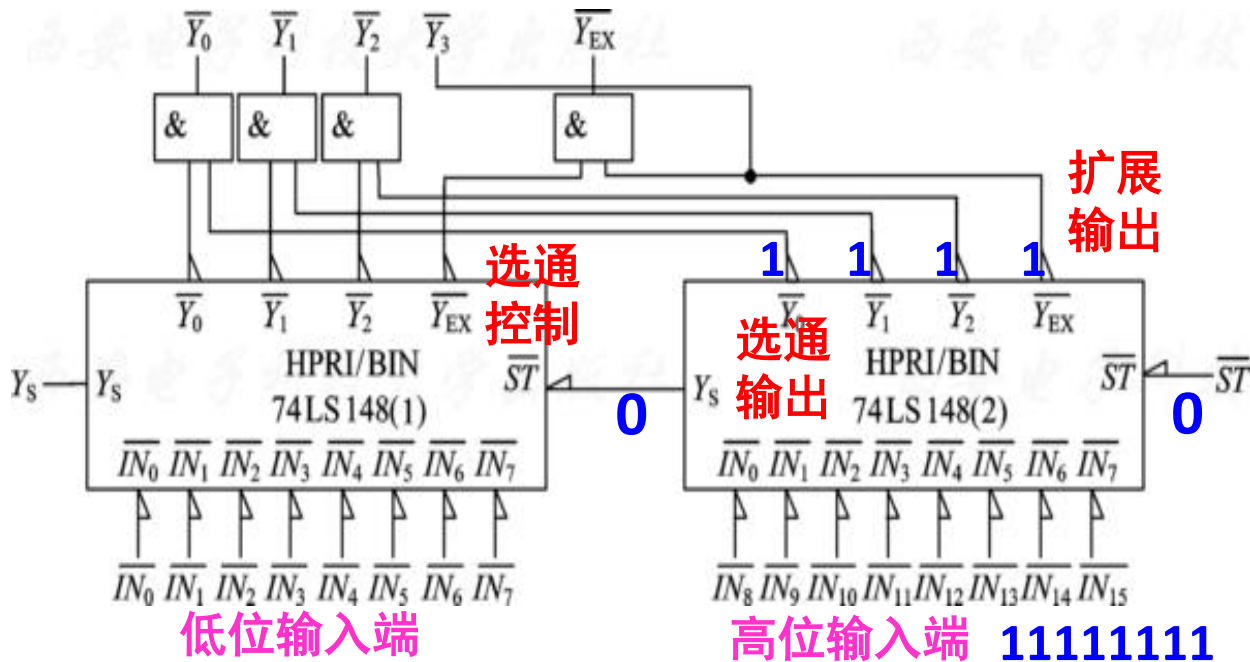


- 系统由2片74LS148构成，低8位输入端接入芯片1，高8位输入端接入芯片2
- 高位选通输出端与低位选通控制端连接

$$\begin{cases} \overline{Y}_3 = \overline{Y_{EX2}} \\ \overline{Y}_2 = \overline{Y_{22}} \overline{Y_{12}} \\ \overline{Y}_1 = \overline{Y_{21}} \overline{Y_{11}} \\ \overline{Y}_0 = \overline{Y_{20}} \overline{Y_{10}} \end{cases}$$

4.4.2 编码器

2. 优先编码器



$\overline{ST}$	$\overline{I_0}$	$\overline{I_1}$	$\overline{I_2}$	$\overline{I_3}$	$\overline{I_4}$	$\overline{I_5}$	$\overline{I_6}$	$\overline{I_7}$	$\overline{Y_2}$	$\overline{Y_1}$	$\overline{Y_0}$	$\overline{Y_{EX}}$	Y_s
1	Φ	Φ	Φ	Φ	Φ	Φ	Φ	Φ	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	Φ	Φ	Φ	Φ	Φ	Φ	Φ	0	0	0	0	0	1
0	Φ	Φ	Φ	Φ	Φ	Φ	0	1	0	0	1	0	1
0	Φ	Φ	Φ	Φ	Φ	0	1	1	0	1	0	0	1
0	Φ	Φ	Φ	Φ	0	1	1	1	0	1	1	0	1
0	Φ	Φ	Φ	0	1	1	1	1	1	0	0	0	1
0	Φ	Φ	0	1	1	1	1	1	1	0	1	0	1
0	Φ	0	1	1	1	1	1	1	1	1	0	0	1
0	0	1	1	1	1	1	1	1	1	1	1	0	1

当芯片2的选通控制端为0，且所有高位输入为1时：

- 芯片2的三个编码输出端均为1：

$$\overline{Y_{22}} = \overline{Y_{21}} = \overline{Y_{20}} = 1$$

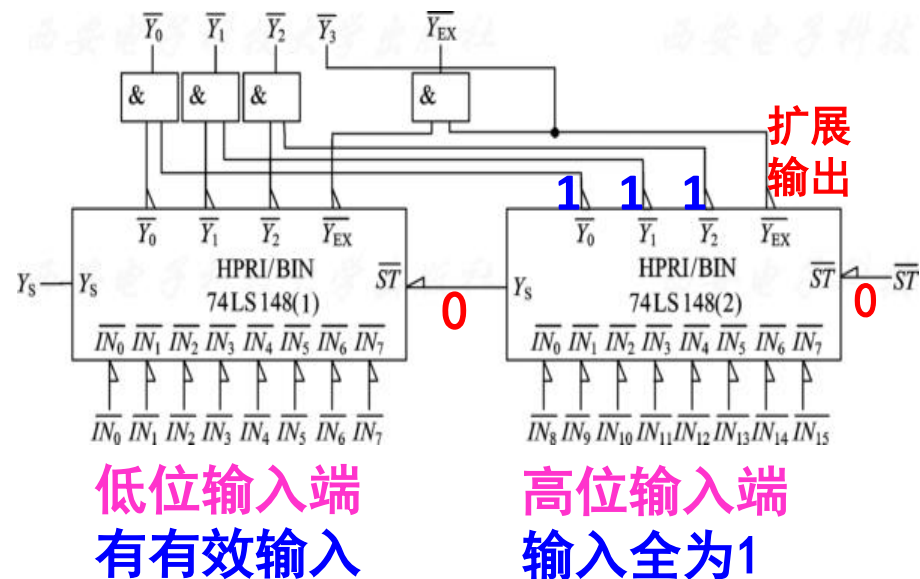
- 芯片2的扩展输出端为1，则整个电路输出的最高位：

$$\overline{Y_3} = \overline{Y_{2EX}} = 1$$

- 芯片2的选通输出为0，则芯片1的选通控制为0，所以芯片1进入工作状态；

4.4.2 编码器

2. 优先编码器



优先编码器74LS148功能表

$\overline{ST}$	$\overline{I_0}$	$\overline{I_1}$	$\overline{I_2}$	$\overline{I_3}$	$\overline{I_4}$	$\overline{I_5}$	$\overline{I_6}$	$\overline{I_7}$	$\overline{Y_2}$	$\overline{Y_1}$	$\overline{Y_0}$	$\overline{Y_{EX}}$	Y_s
1	Φ	Φ	Φ	Φ	Φ	Φ	Φ	Φ	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	Φ	Φ	Φ	Φ	Φ	Φ	Φ	0	0	0	0	0	1
0	Φ	Φ	Φ	Φ	Φ	Φ	0	1	0	0	1	0	1
0	Φ	Φ	Φ	Φ	Φ	0	1	1	0	1	0	0	1
0	Φ	Φ	Φ	Φ	0	1	1	1	0	1	1	0	1
0	Φ	Φ	Φ	0	1	1	1	1	1	0	0	0	1
0	Φ	Φ	0	1	1	1	1	1	1	0	1	0	1
0	Φ	0	1	1	1	1	1	1	1	1	0	0	1
0	0	1	1	1	1	1	1	1	1	1	1	0	1

输入端有有效输入

当芯片1
进入正常
工作状态
时:

➤ 芯片1的三个输出端为最优先有效输出端的编码: $\overline{Y_{12}}, \overline{Y_{11}}, \overline{Y_{10}}$

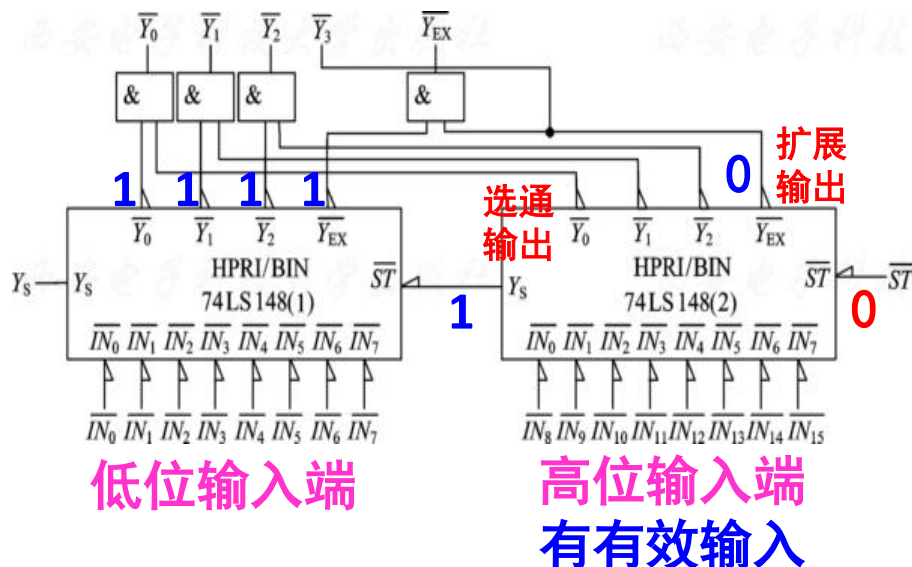
➤ 整个电路的输出为:

$$\begin{cases} \overline{Y_3} = 1 \\ \overline{Y_2} = \overline{Y_{22}} \quad \overline{Y_{12}} = \overline{Y_{12}} \\ \overline{Y_1} = \overline{Y_{21}} \quad \overline{Y_{11}} = \overline{Y_{11}} \\ \overline{Y_0} = \overline{Y_{20}} \quad \overline{Y_{10}} = \overline{Y_{10}} \end{cases}$$

输出值的范围:

1000~1111

2. 优先编码器



优先编码器74LS148功能表

[illegible]

输入端有有效输入

当芯片
2 输入
端有有
效输入
信号时:

- 芯片2的选通输出为1，扩展输出为0；
- 芯片1的选通控制为1，则芯片1不工作，其所有输出均为1；

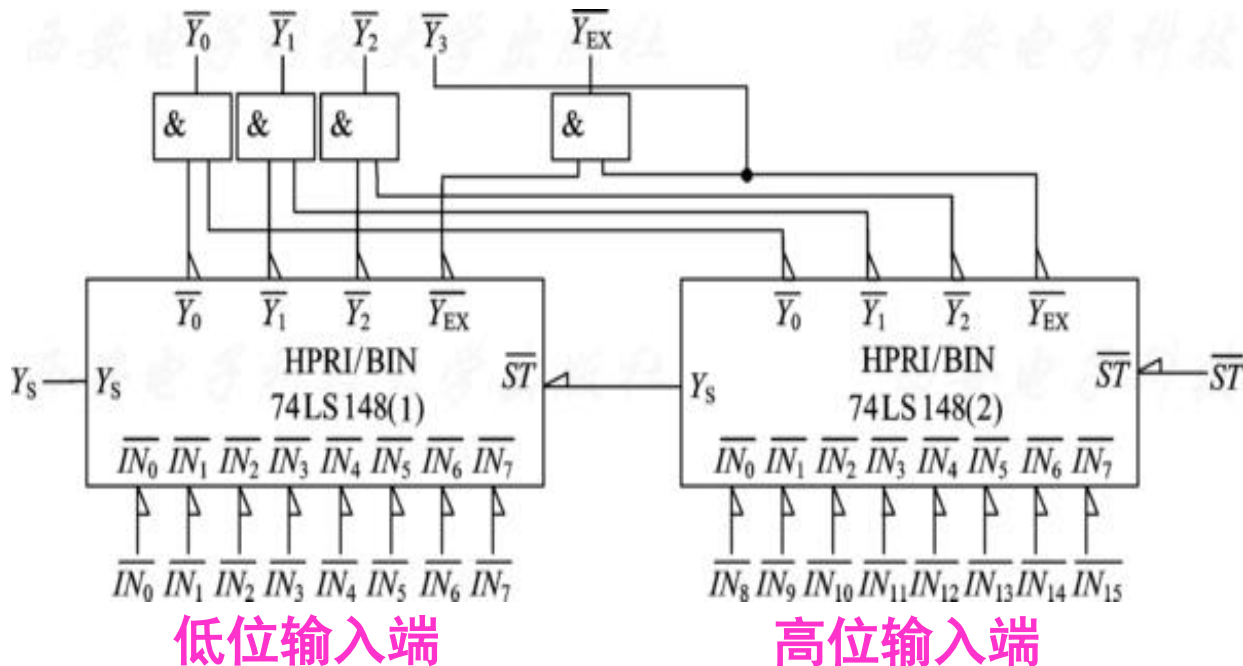
➤ 整个电路的输出为:

$$\left\{ \begin{array}{l} \overline{Y_3} = 0 \\ \overline{Y_2} = \overline{Y_{22}} \quad \overline{Y_{12}} = \overline{Y_{22}} \\ \overline{Y_1} = \overline{Y_{21}} \quad \overline{Y_{11}} = \overline{Y_{21}} \\ \overline{Y_0} = \overline{Y_{20}} \quad \overline{Y_{10}} = \overline{Y_{20}} \end{array} \right.$$

输出值的范围：
0000~0111

4.4.2 编码器

2. 优先编码器



总结:

➤ 当高8位有有效输入时，输出为:

0000—0111

➤ 当高8位无有效输入，低8位有有效输入时，输出为:

1000-1111

主要内容

Main Content

加法器

编码器

译码器

4.4.3 译码器

译码： 将具有特定含义的二进制**代码**变换(翻译)成一定的**输出信号**, 以表示二进制代码的原意, 这一过程称为译码

译码是编码的逆过程, 即将某个二进制代码翻译成电路的某种状态。

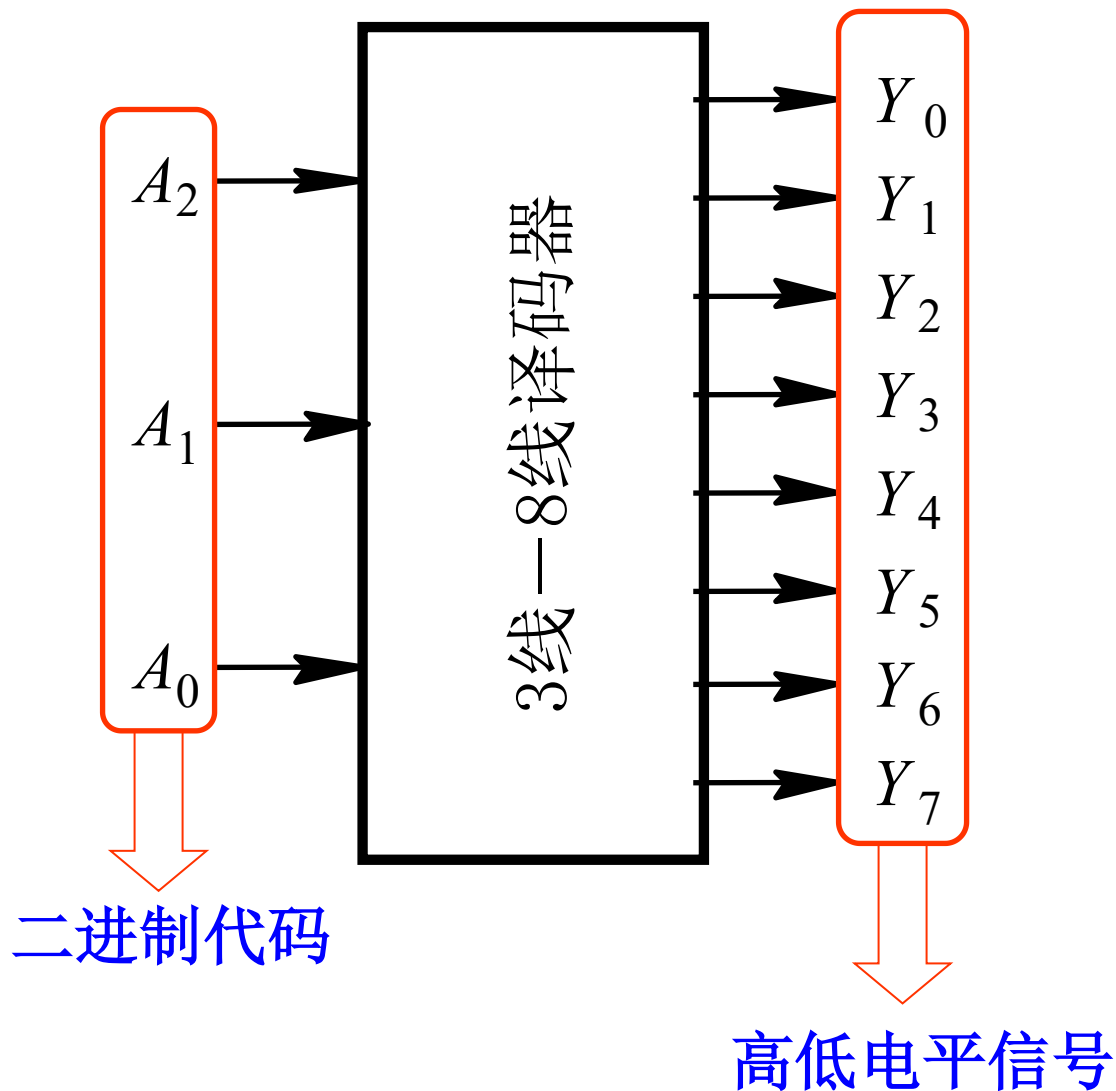
实现译码功能的组合电路称为译码器

- 二进制译码器
- 二—十进制译码器
- 显示译码器

4.4.3 译码器

1. 二进制译码器

- 二进制译码器把输入的 n 位二进制代码翻译为 2^n 个输出的高低电平信号，其中有一个为有效电平，其编号对应于输入的二进制代码
- 常见的译码器有2-4译码器、3-8译码器和4-16译码器。



4.4.3 译码器

1. 二进制译码器

真值表

<i>A</i>	<i>B</i>	<i>C</i>	<i>Y</i> ₀	<i>Y</i> ₁	<i>Y</i> ₂	<i>Y</i> ₃	<i>Y</i> ₄	<i>Y</i> ₅	<i>Y</i> ₆	<i>Y</i> ₇
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

表达式

$$Y_0 = \overline{A} \overline{B} \overline{C}$$

$$Y_1 = \overline{A} \overline{B} C$$

$$Y_2 = \overline{A} B \overline{C}$$

•

•

$$Y_7 = A B C$$

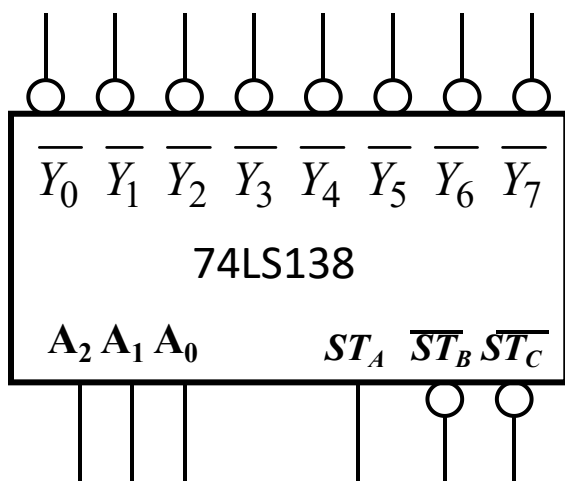
说明 — 译码器是多输入、多输出组合逻辑电路，每个输出对应一个n变量最小项——也称**最小项发生器**。

4.4.3 译码器

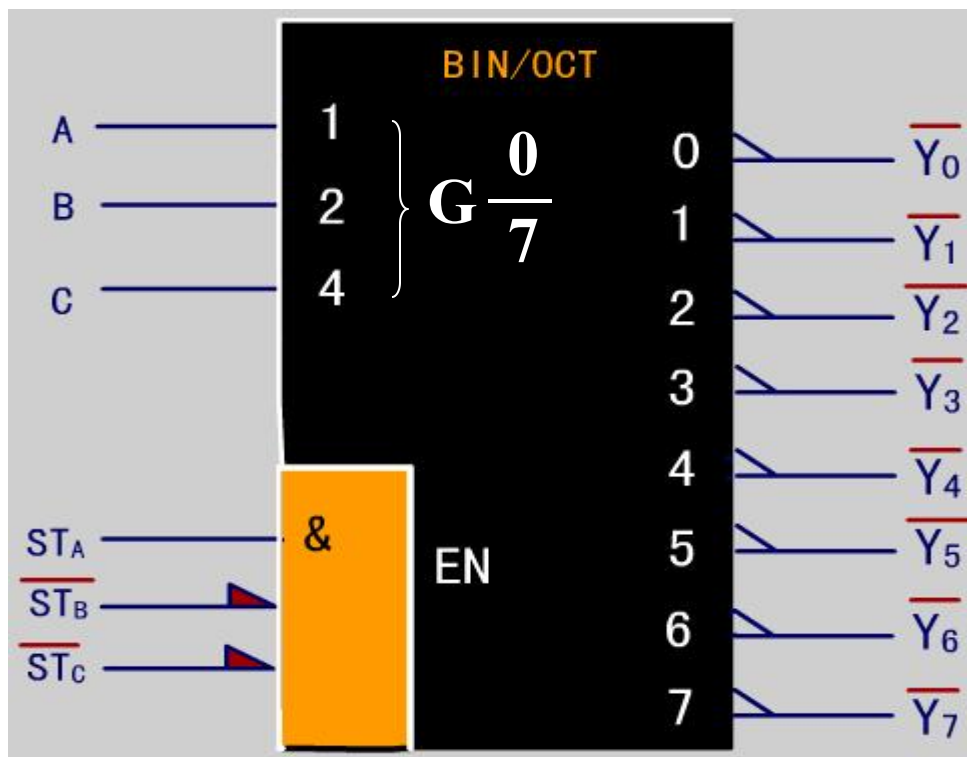
1. 二进制译码器

74LS138 (3/8译码器)

一般符号



图形符号



1. 二进制译码器

74LS138功能表

[illegible]

- 3个输入端：
A, B, C
- 8个输出端：
 $\overline{Y}_0 \dots \overline{Y}_7$ (低有效)
- 3个使能端：
 $ST_A, \overline{ST}_B, \overline{ST}_C$

4.4.3 译码器

1. 二进制译码器

3-8译码器的 Verilog HDL 实现代码

```
module decoder(iSTA, iSTB_N, iSTC_N, iA, oY_N);  
    input iSTA, iSTB_N, iSTC_N;  
    input [2:0] iA;  
    output [7:0] oY_N;  
    reg [7:0] m_y;  
    assign oY_N=m_y;  
    always@(iSTA, iSTB_N, iSTC_N, iA)  
        if(iSTA && !(iSTB_N||iSTC_N))  
            case(iA)  
                3'b000: m_y = 8'b01111111;  
                3'b001: m_y = 8'b10111111;  
                3'b010: m_y = 8'b11011111;  
                3'b011: m_y = 8'b11101111;  
                3'b100: m_y = 8'b11110111;  
                3'b101: m_y = 8'b11111011;  
                3'b110: m_y = 8'b11111101;  
                3'b111: m_y = 8'b11111110;  
            endcase  
        else  
            m_y=8'hff;  
        endmodule
```

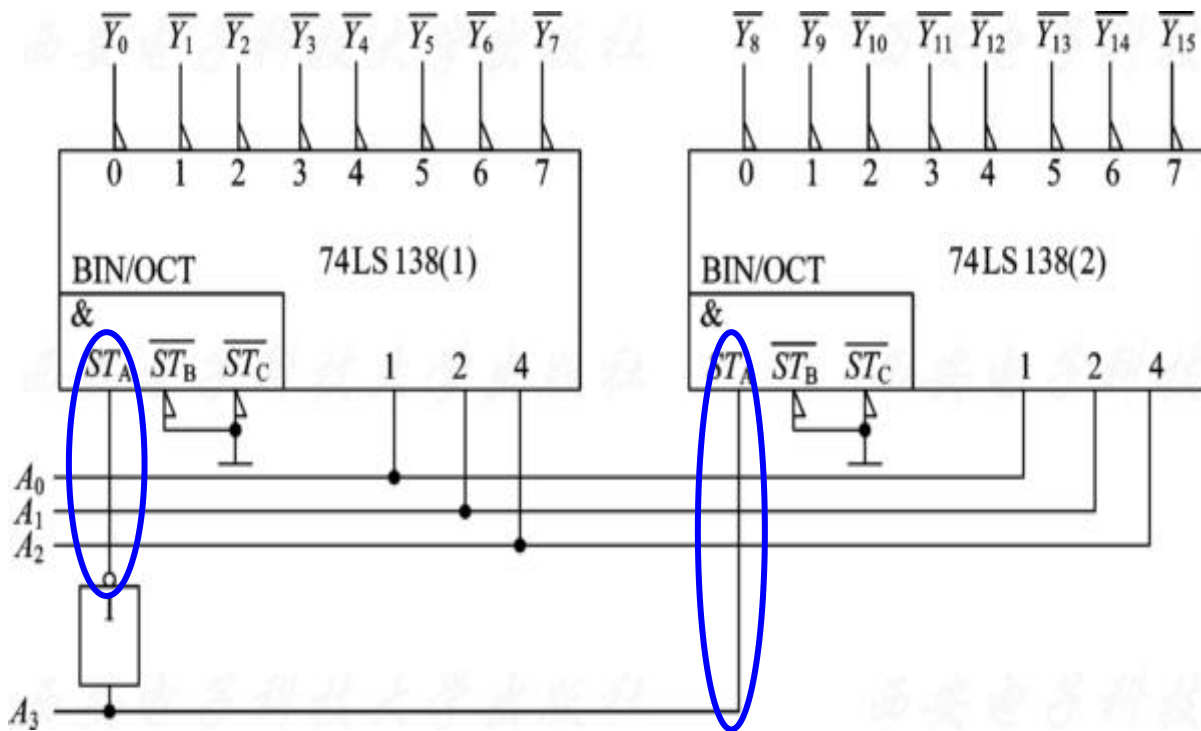
4.4.3 译码器

1. 二进制译码器

两片74LS138构成4-16译码器

低位输出

高位输出



$A_3A_2A_1A_0$: 译码输入

$A_3=0$ 时，片1工作，片2禁止。

$A_3A_2A_1A_0$: 0000-0111

有效输出产生于:
 $\overline{Y_0} \sim \overline{Y_7}$

$A_3=1$ 时，片2工作，片1禁止

$A_3A_2A_1A_0$: 1000-1111

有效输出产生于: $\overline{Y_8} \sim \overline{Y_{15}}$

用最高位地址
作为片选信号

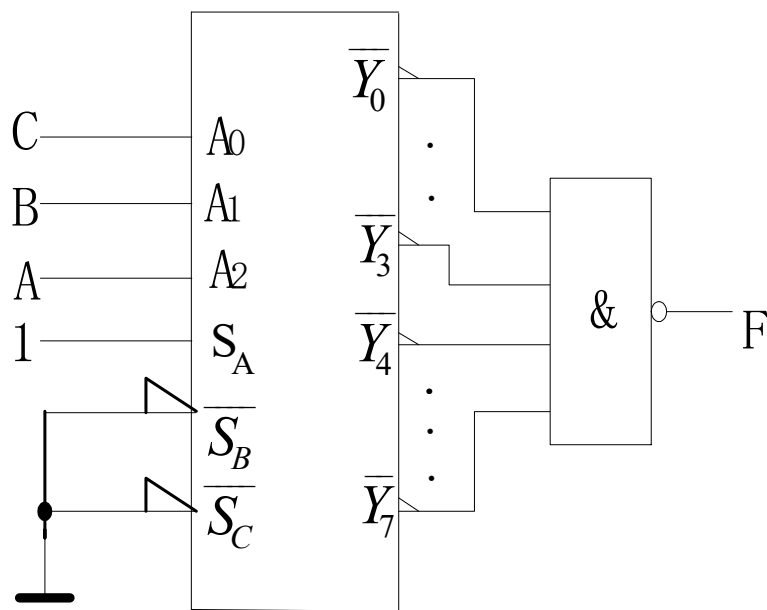
4.4.3 译码器

1. 二进制译码器

用74LS138实现函数

译码器的输出分别对应一个**最小项**（**高电平译码**）或一个**最小项的非**（**低电平译码**），所以附加适当门，可实现任意函数。

特点：方法简单，无须简化，工作可靠。



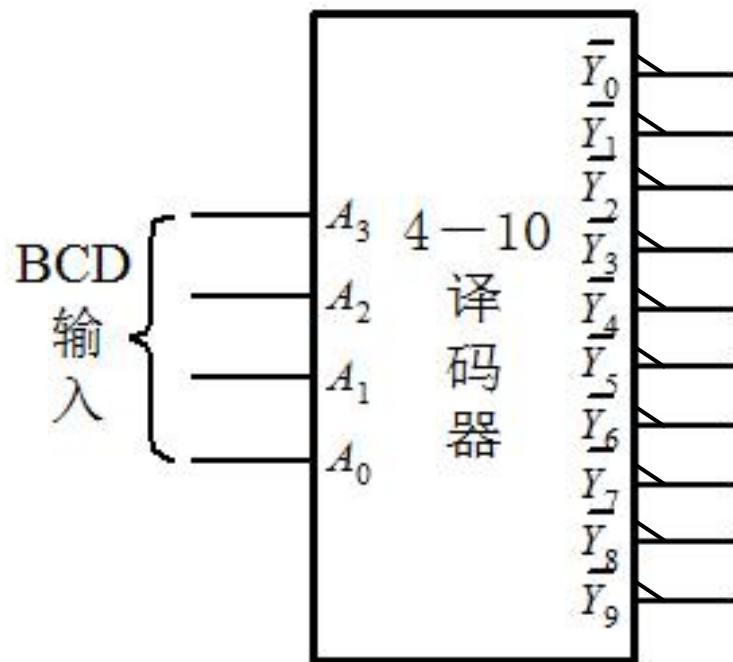
$$\begin{aligned}
 F &= \overline{\overline{Y_0} \overline{Y_3} \overline{Y_4} \overline{Y_7}} \\
 &= Y_0 + Y_3 + Y_4 + Y_7 \\
 &= m_0 + m_3 + m_4 + m_7 \\
 &= \sum (0, 3, 4, 7)
 \end{aligned}$$

4.4.3 译码器

2. 二—十进制译码器

- 二—十进制译码器把输入的4位BCD码翻译为10个输出的高低电平信号，其中有一个为有效电平，其编号对应于输入的BCD码

二—十进制译码器74LS42逻辑图



4.4.3 译码器

2. 二—十进制译码器

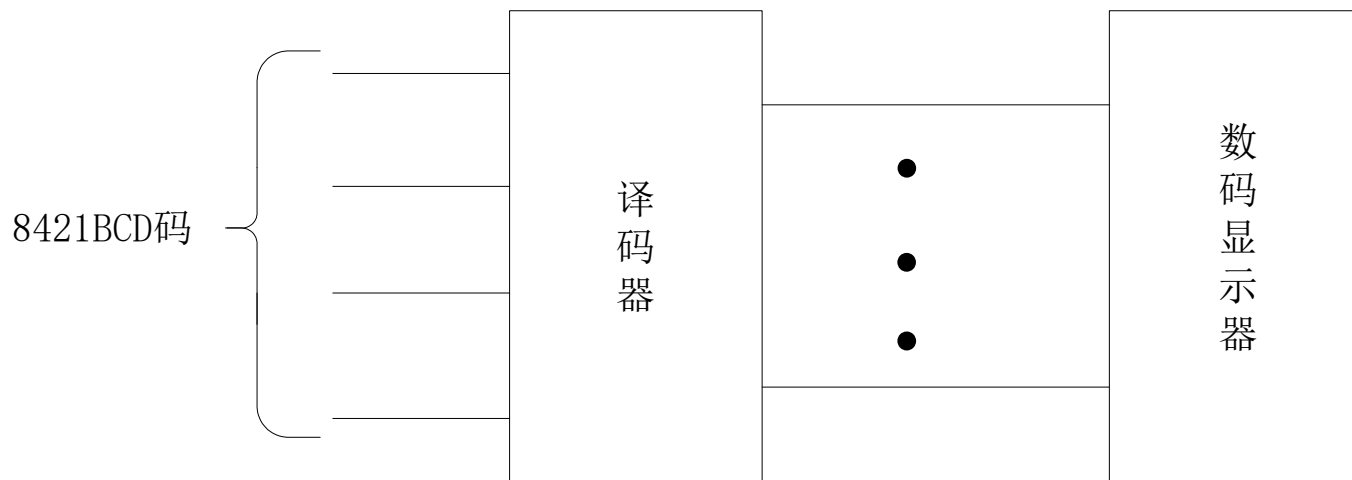
74LS42的真值表

A ₃ A ₂ A ₁ A ₀	$\overline{Y_0}$	$\overline{Y_1}$	$\overline{Y_2}$	$\overline{Y_3}$	$\overline{Y_4}$	$\overline{Y_5}$	$\overline{Y_6}$	$\overline{Y_7}$	$\overline{Y_8}$	$\overline{Y_9}$
0 0 0 0	0	1	1	1	1	1	1	1	1	1
0 0 0 1	1	0	1	1	1	1	1	1	1	1
0 0 1 0	1	1	0	1	1	1	1	1	1	1
0 0 1 1	1	1	1	0	1	1	1	1	1	1
0 1 0 0	1	1	1	1	0	1	1	1	1	1
0 1 0 1	1	1	1	1	1	0	1	1	1	1
0 1 1 0	1	1	1	1	1	1	0	1	1	1
0 1 1 1	1	1	1	1	1	1	1	0	1	1
1 0 0 0	1	1	1	1	1	1	1	1	0	1
1 0 0 1	1	1	1	1	1	1	1	1	1	0
1 0 1 0	1	1	1	1	1	1	1	1	1	1
⋮										
1 1 1 1	1	1	1	1	1	1	1	1	1	1

冗余输入,
 无有效输出

3. 数字显示译码器

在数字系统中，常需把结果用十进制数码显示出来，数字显示电路包括**译码驱动电路**和**数码显示器**。



8421BCD显示译码电路框图

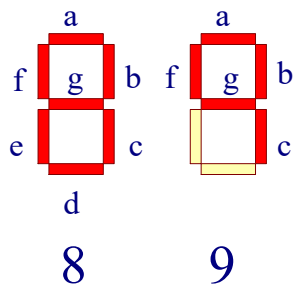
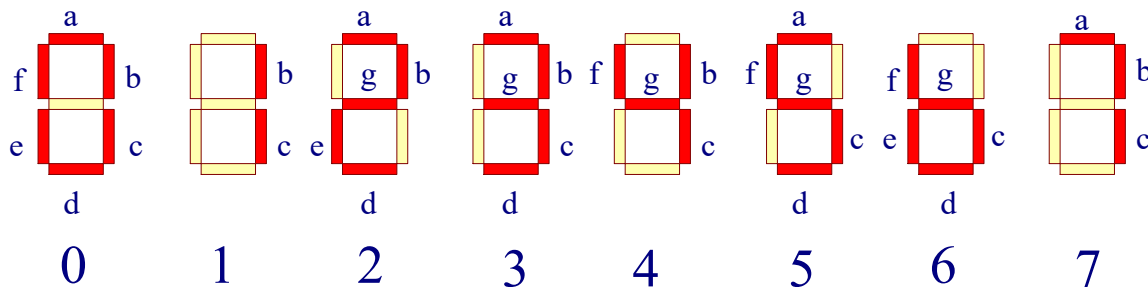
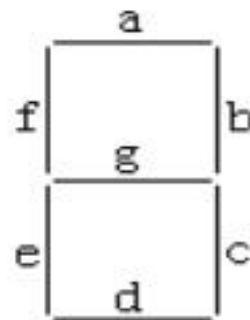
3. 数字显示译码器

(1) 七段数码管

每一段由一个发光二极管组成

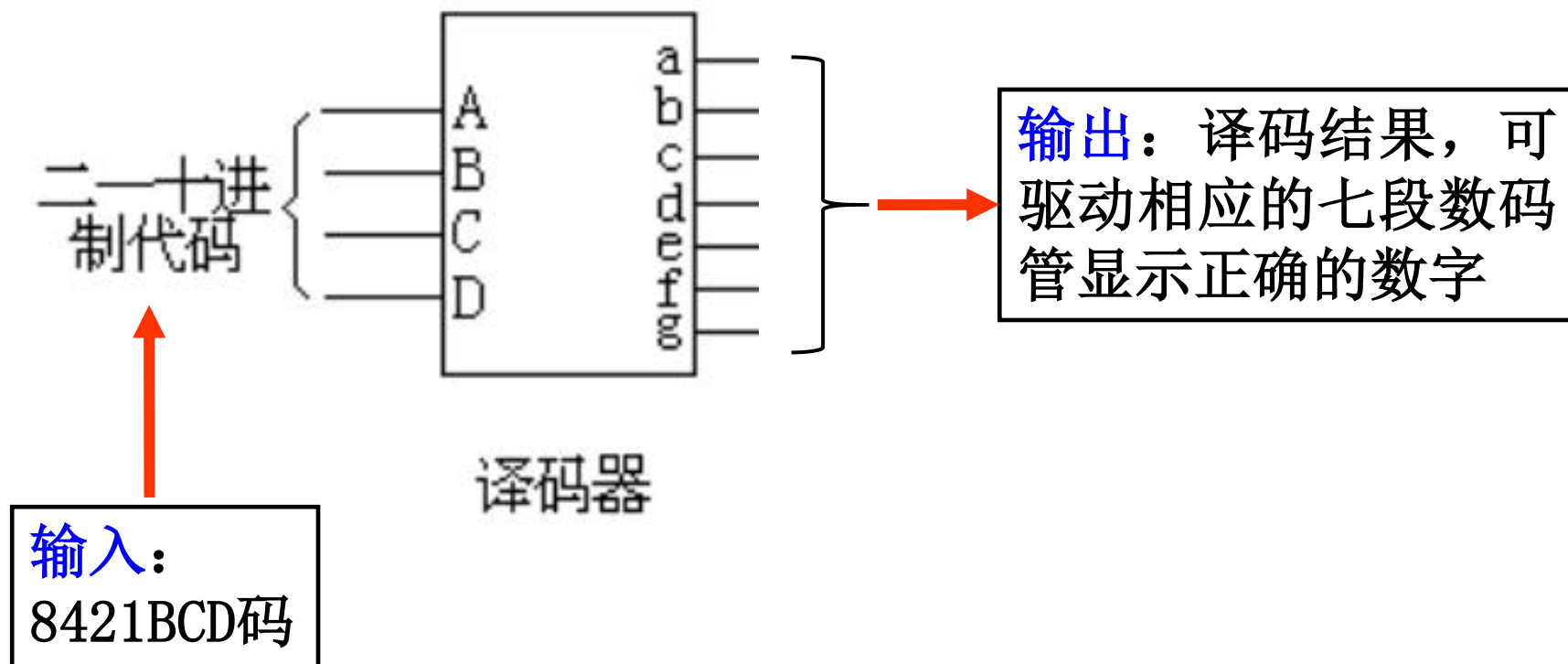
共阴极： 高电平亮

共阳极： 低电平亮



3. 数字显示译码器

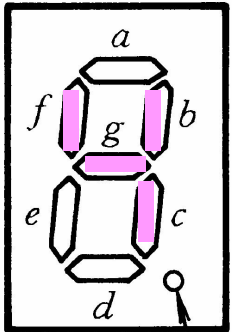
(2) 七段显示译码器



4.4.3 译码器

3. 数字显示译码器

BCD七段字符显示译码器74LS48

输 入					输 出							字形
数字	A ₃	A ₂	A ₁	A ₀	Y _a	Y _b	Y _c	Y _d	Y _e	Y _f	Y _g	
0	0	0	0	0	1	1	1	1	1	1	0	
1	0	0	0	1	0	1	1	0	0	0	0	
2	0	0	1	0	1	1	0	1	1	0	1	
3	0	0	1	1	1	1	1	1	0	0	1	
4	0	1	0	0	0	1	1	0	0	1	1	
5	0	1	0	1	1	0	1	1	0	1	1	
6	0	1	1	0	0	0	1	1	1	1	1	
7	0	1	1	1	1	1	1	0	0	0	0	
8	1	0	0	0	1	1	1	1	1	1	1	
9	1	0	0	1	1	1	1	0	0	1	1	

4.4.3 译码器

3. 数字显示译码器

具有共阴共阳输出可选控制的七段译码模块

```
module samp4_4_7(flag,A,Y);  
    input flag;  
    input[3:0] A;  
    output reg [6:0] Y;  
    seg_decoder u1 (.iflag(flag), .iA(A), .oY(Y));
```

Endmodule

```
module seg_decoder (iflag, iA, oY);  
    //七段译码模块定义  
    input iflag; //共阴、共阳输出控制端,  
    input[3:0] iA; //四位二进制输入  
    output reg [6:0] oY;  
    always@(iflag,iA)  
        begin  
            case(iA)  
                4'b0000: oY=7'h3f; //iflag=1共阴极输出  
                4'b0001: oY=7'h06;  
                4'b0010: oY=7'h5b;  
                4'b0011: oY=7'h4f;
```

```
                4'b0100: oY=7'h66;  
                4'b0101: oY=7'h6d;  
                4'b0110: oY=7'h7d;  
                4'b0111: oY=7'h27;  
                4'b1000: oY=7'h7f;  
                4'b1001: oY=7'h6f;  
                4'b1010: oY=7'h77;  
                4'b1011: oY=7'h7c;  
                4'b1100: oY=7'h58;  
                4'b1101: oY=7'h5e;  
                4'b1110: oY=7'h79;  
                4'b1111: oY=7'h71;  
            endcase  
            if(!iflag)  
                oY=~oY; //iflag=0共阳极输出  
            end  
        endmodule
```

4. 译码器应用

【例】 用译码器实现下面两个函数。

$$F_1 = \overline{A}\overline{B} + \overline{B}C + AC$$

$$F_2 = \overline{A}\overline{B} + B\overline{C} + ABC$$

解：这两个输出函数均为三输入变量函数，下面分别用3-8译码器+与非门和3-8译码器+与门实现F1、F2。

方法一： 选用3-8译码器和与非门实现。

4. 译码器应用

将输出函数写成最小项之和的形式，并变换为译码器反码输出形式，用与非门作为F1、F2的输出门。

$$\begin{aligned}
 F_1(A, B, C) &= \overline{A}B + \overline{B}C + AC \\
 &= \overline{A}\overline{B}(C + \overline{C}) + \overline{B}C(A + \overline{A}) + AC(B + \overline{B}) \\
 &= \overline{A}\overline{B}C + \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + \overline{A}BC \\
 &= m_1 + m_4 + m_5 + m_7 \\
 &= \overline{\overline{m_1 + m_4 + m_5 + m_7}} \\
 &= \overline{\overline{m_1} \cdot \overline{m_4} \cdot \overline{m_5} \cdot \overline{m_7}} \\
 &= \overline{Y_1 \cdot Y_4 \cdot Y_5 \cdot Y_7}
 \end{aligned}$$

4.4.3 译码器

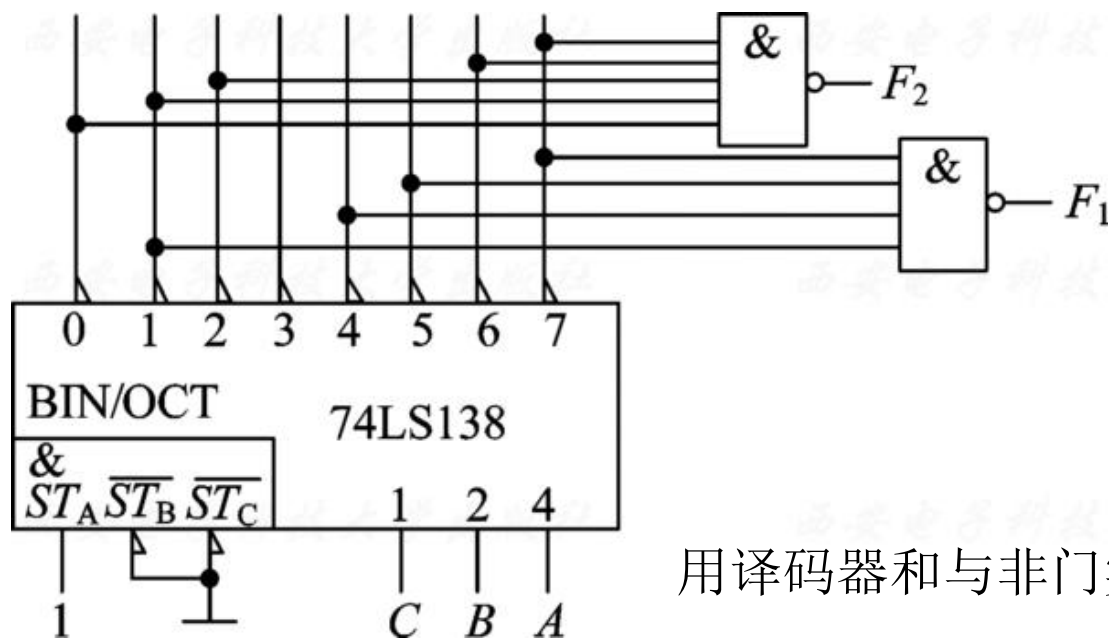
4. 译码器应用

$$F_2(A, B, C) = \overline{A}\overline{B} + B\overline{C} + ABC$$

$$= m_0 + m_1 + m_2 + m_6 + m_7$$

$$= \overline{\overline{m_0} \cdot \overline{m_1} \cdot \overline{m_2} \cdot \overline{m_6} \cdot \overline{m_7}}$$

$$= \overline{Y_0 \cdot Y_1 \cdot Y_2 \cdot Y_6 \cdot Y_7}$$



用译码器和与非门实现

4. 译码器应用

方法二： 选用3-8译码器和与门实现。

将输出函数写成最大项之积的形式， 然后进行如下变换：

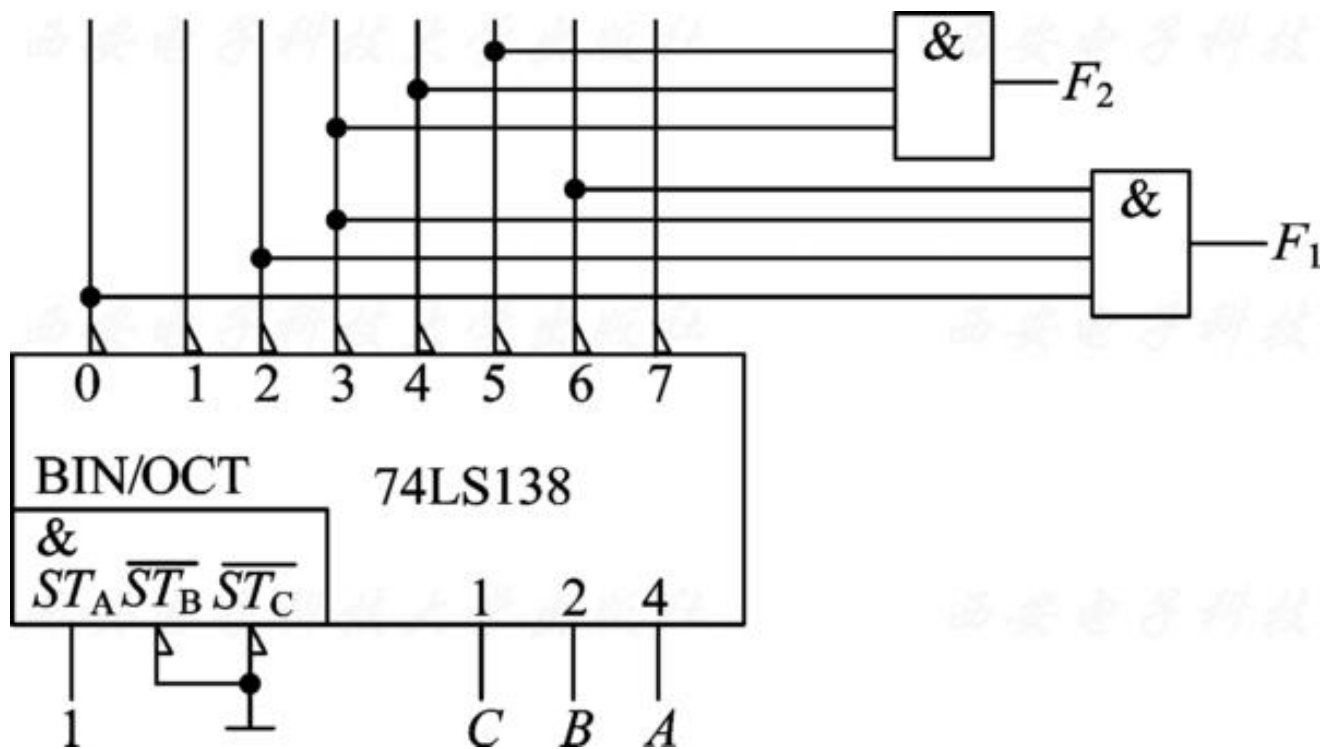
$$\begin{aligned} F_1(A, B, C) &= \overline{A}\overline{B} + \overline{B}C + AC \\ &= m_1 + m_4 + m_5 + m_7 \\ &= \sum m(1, 4, 5, 7) \\ &= \prod M(0, 2, 3, 6) \\ &= M_0 \cdot M_2 \cdot M_3 \cdot M_6 \\ &= \overline{Y}_0 \cdot \overline{Y}_2 \cdot \overline{Y}_3 \cdot \overline{Y}_6 \end{aligned}$$

4. 译码器应用

$$\begin{aligned} F_2(A, B, C) &= \overline{A}\overline{B} + B\overline{C} + ABC \\ &= m_0 + m_1 + m_2 + m_6 + m_7 \\ &= \sum m(0, 1, 2, 6, 7) \\ &= \prod M(3, 4, 5) \\ &= M_3 \cdot M_4 \cdot M_5 \\ &= \overline{Y}_3 \cdot \overline{Y}_4 \cdot \overline{Y}_5 \end{aligned}$$

4.4.3 译码器

4. 译码器应用



用译码器和与门实现

第四章习题

P168 #9 用for循环语句编写8线-3线优先编码器的Verilog代码。

P168 #18 用74LS138和必要的门电路实现下列逻辑

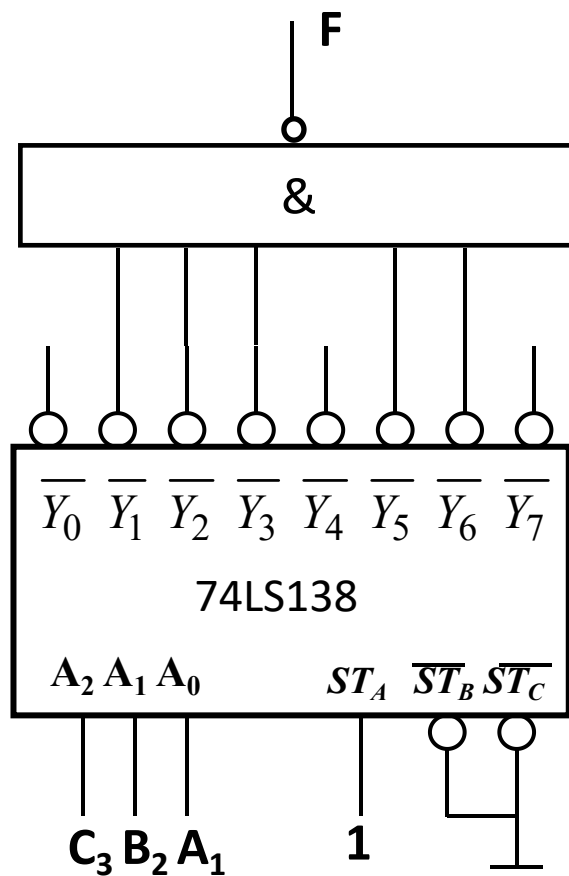
函数: (1) $f(A_1, B_2, C_3) = \sum m(0, 2, 3, 4, 5, 7)$

(2) $f(A_1, B_2, C_3) = \sum m(1, 2, 3, 5, 6)$

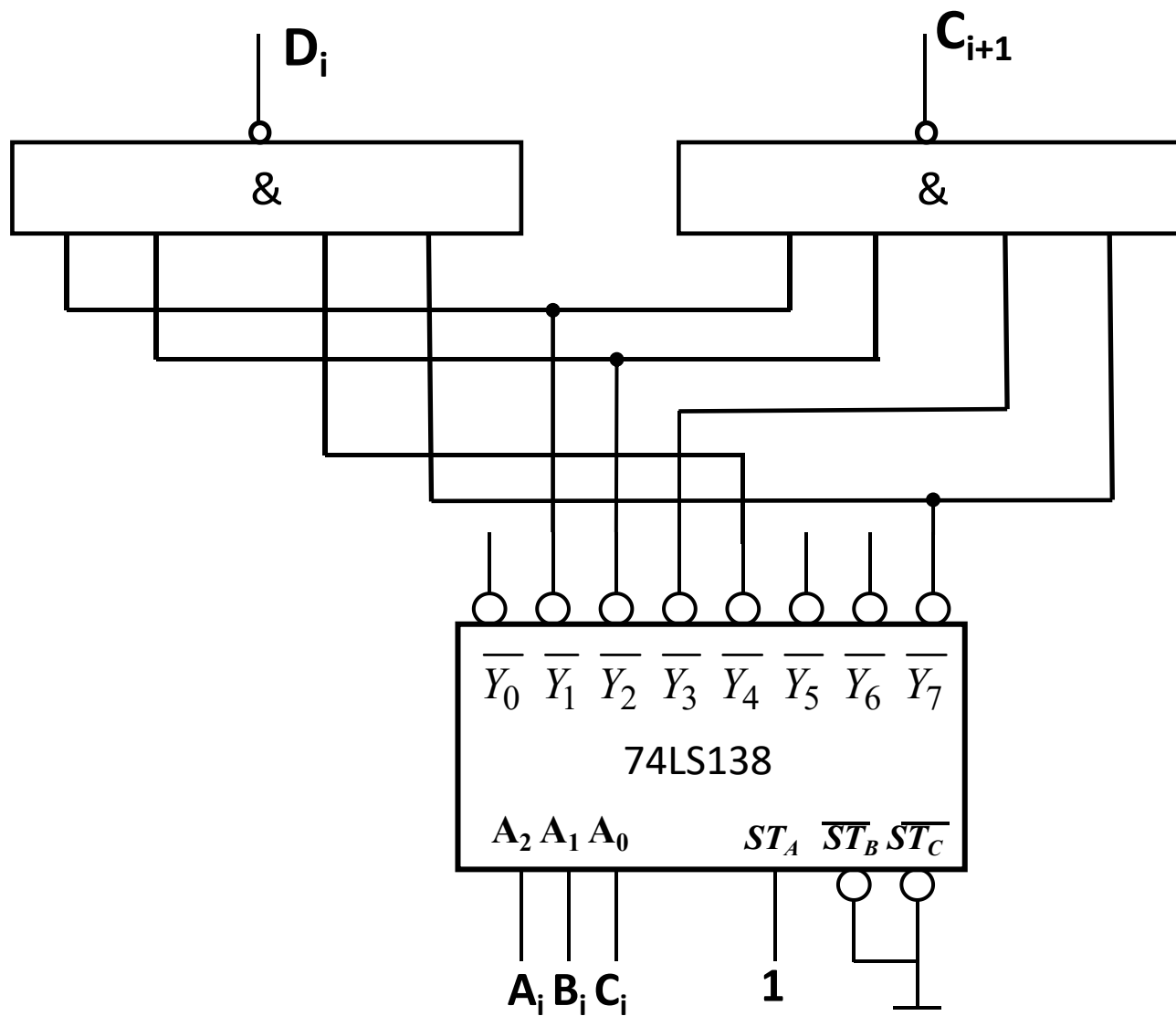
4.4 常用组合逻辑电路

$$(1) f(A_1, B_2, C_3) = \sum m(0, 2, 3, 4, 5, 7)$$

$$(2) f(A_1, B_2, C_3) = \sum m(1, 2, 3, 5, 6)$$



4.4 常用组合逻辑电路





武汉大学
Wuhan University

谢谢!

2025-11-20

